Ficha 8. Guardar, compartir y llevar a la mesa

Taller virtual ATA EPE · Módulo 8: compartir

Meta del módulo. Que un montaje se pueda guardar, abrir en otro aparato y reproducir igual; compartirlo con un enlace; proponer retos (armar los siete prototipos de la cartilla) que el simulador sabe verificar; y exportar lo armado, con su movimiento, a realidad aumentada. Cara pública: **compartir y retos**.

Lecturas

1. **Especificación glTF 2.0** (Khronos Group): nodos, mallas y animaciones; el formato que ya usa la web.
2. **Goldberg, «What every computer scientist should know about floating-point arithmetic»**, *ACM Computing Surveys* 23(1), 1991: por qué el determinismo es delicado.
3. Documentación de `<model-viewer>` (realidad aumentada) y de `CompressionStream` (navegador).
4. Lo hecho en `herramientas/generar_mecanismos.py`: ya muestrea animaciones a 30 cuadros por segundo y las exporta a GLB y USDZ.

1. El montaje como documento

Un montaje se guarda como su **grafo de ensamble**, no como el estado de la simulación: piezas (catálogo y motor), uniones (qué tornillo, qué agujeros, estado de la tuerca), bandas y parámetros. Con un número de versión, para que los montajes viejos se sigan abriendo cuando el formato cambie.

```
{ "version": 1,
  "piezas": [ { "id": "viga-9", "motor": [1, 0, 0, ...] }, ... ],
  "uniones": [ { "tornillo": 3, "piezas": [0, 1], "tuerca": "floja" }, ... ],
  "bandas": [ { "poleas": [4, 5], "cruzada": false } ] }
```

Ejercicio 8.1

Diseña el formato completo. ¿Qué información se puede *reconstruir* al abrir (masas, cuerpos compuestos, formas de colisión) y por tanto no se guarda? ¿Cómo se valida un archivo dañado o malicioso?

2. Determinismo

Mismo montaje, mismo paso de tiempo y mismo orden de operaciones dan el mismo resultado *en el mismo motor de JavaScript*. Entre aparatos hay una trampa: la norma IEEE 754 garantiza que $+$, $-$, $\times$, $\div$ y la raíz cuadrada den exactamente el mismo resultado en todas partes, pero *no* las funciones trascendentes (`Math.sin`, `Math.exp`), que pueden diferir en el último bit. En un sistema caótico (una biela que se traba o no), esa diferencia crece.

Ejercicio 8.2

Propón dos soluciones: (a) implementar seno, coseno y exponencial propios con solo las operaciones garantizadas; (b) guardar «fotos» del estado cada cierto tiempo. ¿Qué cuesta cada una? ¿Cuál basta para compartir un montaje?

3. Compartir con un enlace

El documento JSON se comprime (`CompressionStream('deflate-raw')`), se codifica en base64 apto para URL y va en el fragmento del enlace (`#m=...`), que no se envía al servidor. Un montaje típico ocupa unos cientos de bytes: cabe en un código QR.

4. Retos verificables

Cada reto es un predicado sobre el montaje, calculado con lo construido en los módulos anteriores:

- «Arma la biela-manivela»: un grado de libertad (módulo 3) y la trayectoria del pescador sube y baja.
- «Haz que la rueda grande gire tres veces más lento»: relación de transmisión 1/3 medida (módulo 6).
- «Equilibra la balanza con pesas distintas»: $m_1 x_1 = m_2 x_2$ por trabajo virtual (módulo 7).

Un reto no compara con una solución guardada: comprueba la física. Así hay muchas formas de resolverlo.

5. A la mesa: realidad aumentada

Se simula el montaje unos segundos, se muestrea el motor de cada pieza a 30 cuadros por segundo y se escribe un GLB con una pista de animación por pieza (traslación y rotación de cada motor), como hace `generar_mecanismos.py`. En Android, `<model-viewer>` lo muestra en la mesa con movimiento. En iPhone, Quick Look necesita USDZ, y el exportador del navegador no incluye animación: por ahora el montaje se ve quieto.

Bucle sin salto

Para que la animación se repita sin salto, se muestrea un número entero de periodos del mecanismo: el periodo de la manivela, o el mínimo común múltiplo cuando hay varias ruedas (como en los mecanismos de la web).

Qué programar

`app/archivo.js`	Guardar y abrir montajes (JSON con versión y validación); enlace comprimido; QR.
`app/retos.js`	Retos como predicados físicos; los siete prototipos de la cartilla.
`app/exportar.js`	Muestreo de la simulación y GLB animado; visor de realidad aumentada.
`pruebas/`	Guardar y abrir da el mismo grafo; misma simulación en el mismo motor; retos resueltos y no resueltos.

Preguntas de control

1. ¿Por qué se guarda el grafo de ensamble y no el estado de la simulación?
2. ¿Qué garantiza IEEE 754 y qué no, y por qué importa en un sistema caótico?
3. ¿Por qué un reto debe comprobar la física y no comparar con una solución?
4. ¿Cómo se consigue que la animación exportada se repita sin salto?

Cierre del módulo

Ejercicios 8.1–8.2 · pruebas de archivo, enlace y retos · los siete prototipos de la cartilla armados y verificados en el taller · cara pública: compartir, retos y realidad aumentada. **Fin del curso.**